

Actividades

1. Escriba una función que reutilice la función de colapso de SML (*foldr*). Esta función debe tomar una lista de números enteros y de' como resultado el numero mas grande (el máximo).

```
getMaxNumber [~45,34,~90,12,34,67];  
>67 :int
```

```
SML  
1 exception ListaVacía;  
2  
3 fun getMaxNumber [] = raise ListaVacía  
4   | getMaxNumber (x::xs) =  
5     foldr (fn (n, m) => if n > m then n else m) x xs;  
6  
7 getMaxNumber [~45, 34, ~90, 12, 34, 67];  
8  
  
Output  
> val ListaVacía = ListaVacía: exn;  
> val getMaxNumber = fn: int list → int;  
> val it = 67: int;
```

La función getMaxNumber reutiliza foldr para recorrer la lista sin escribir una recursión explícita completa. La idea es comenzar con el primer elemento como mejor candidato a máximo y luego ir comparándolo con cada valor restante. Si el valor actual es mayor que el acumulado, entonces pasa a ser el nuevo máximo, de lo contrario, se conserva el acumulado. Este enfoque es correcto porque al terminar el recorrido se habrá comparado cada elemento de la lista exactamente una vez. Además, manejar el caso de lista vacía con una excepción evita devolver un valor incorrecto o arbitrario.

2. Escriba una función que reutilice la función de colapso de SML (*foldr*). Esta función debe tomar una lista de números enteros y de' como resultado el numero más pequeño (el mínimo).

```
getMinNumber [~45,34,~90,12,34,67];  
>~90 :int
```

```
SML
1 | exception ListaVacia;
2
3 | fun getMinNumber [] = raise ListaVacia
4 |   getMinNumber (x::xs) =
5 |     foldr (fn (n, m) => if n < m then n else m) x xs;
6
7 | getMinNumber [~45, 34, ~90, 12, 34, 67];|
8

Output
> val ListaVacia = ListaVacia: exn;
> val getMinNumber = fn: int list -> int;
> val it = ~90: int;
```

La función `getMinNumber` sigue la misma estrategia funcional que la del máximo, pero ahora conserva el menor valor encontrado. Se utiliza `foldr` para colapsar la lista en un solo entero, comparando cada elemento con el acumulado parcial. Igual que en la anterior, el primer elemento se usa como base inicial, porque no existe un “mínimo neutro” universal para cualquier lista de enteros. A medida que avanzamos el recorrido, el acumulador siempre representa el menor valor visto hasta ese momento. Al finalizar, el resultado es el mínimo total de la lista.

3. Escriba una función de apareamiento de listas (*zip*) que tome dos listas de la misma longitud y produzca una lista de pares ordenados donde la primera componente proviene la de la primera lista y la segunda componente proviene la segunda lista

```
zipList [1,2,3,4,5] ["a", "b", "c", "d", "e"];
>[(1,"a"), (2,"b"), (3,"c"), (4,"d"), (5,"e")] : (int*string) list
```

```
SML
1 | exception LongitudDistinta;
2
3 | fun zipList [] [] = []
4 |   zipList (x::xs) (y::ys) = (x, y) :: zipList xs ys
5 |   zipList _ _ = raise LongitudDistinta;
6
7 | zipList [1,2,3,4,5] ["a","b","c","d","e"];
```

Output

```
> val LongitudDistinta = LongitudDistinta: exn;  
> val zipList = fn: ∀ 'a 'b . 'a list → 'b list → ('a * 'b) list;  
> val it = [(1, "a"), (2, "b"), (3, "c"), (4, "d"), (5, "e")]: (int * string) list;
```

La función `zipList` toma dos listas de la misma longitud y construye una nueva lista formada por pares ordenados. En cada llamada recursiva, se toma el primer elemento de la primera lista y el primer elemento de la segunda lista, y con ellos se forma un par. Luego la función continúa con las colas de ambas listas hasta llegar al caso base, donde ambas están vacías. Esta forma de resolverlo es el *pattern matching* que permite separar cabeza y cola de manera clara. Si las listas no tienen la misma longitud, se lanza una excepción para indicar que el apareamiento no puede completarse correctamente.

4. Escriba el código de la función de des-apareamiento de listas (*unzip*) que toma como parámetro una lista de pares ordenados y devuelve un par ordenado con dos listas (inversamente a la función anterior)

```
unzipList [(1,2.0),(2,3.0),(3,4.0),(5,6.0)];  
>([1,2,3,5] [2.0, 3.0, 4.0, 6.0]): int list * real list
```

SML

```
1 fun unzipList [] = ([], [])  
2 | unzipList ((x, y)::xs) =  
3   let  
4     val (lx, ly) = unzipList xs  
5   in  
6     (x::lx, y::ly)  
7   end;  
8  
9 unzipList [(1,2.0),(2,3.0),(3,4.0),(5,6.0)];  
10
```

Output

```
> val unzipList = fn: ∀ 'a 'b . ('a * 'b) list → 'a list * 'b list;  
> val it = ([1, 2, 3, 5], [2.0, 3.0, 4.0, 6.0]): int list * real list;
```

La función `unzipList` recibe una lista cuyos elementos son pares y separa sus componentes en dos listas distintas. El caso base ocurre cuando la lista está vacía, y en ese caso el resultado es el par `([],[])`. En el caso recursivo, se toma el

primer par (x,y) y se procesa el resto de la lista para obtener dos listas parciales. Después, x se agrega al frente de la primera lista y y al frente de la segunda. De esta manera, la estructura original de la lista de pares se divide ordenadamente en dos secuencias paralelas. Es exactamente la operación inversa de zipList. 

5. Dado el tipo de dato recursivo correspondiente al árbol

```
Datatype tree =  
Leaf | Br of tree * string * tree
```

Podemos usar el mismo para representar arboles de expresión correspondientes a expresiones aritméticas

```
(x*45)+z/(y+5050)  
Br(  
    Br(  
        Br(Leaf, "x", Leaf),  
        "*",  
        Br(Leaf, "45", Leaf)  
    ),  
    "+",  
    Br(Leaf, "z", Leaf)  
),  
    "/",  
    Br(Leaf, "y", Leaf),  
    "+",  
    Br(Leaf, "5050", Leaf)  
)
```

Realice lo que se pide a continuación:

- a) Construya el código en SML que tome una expresión de *tree* como parámetro y produzca la cadena que representa la serialización en **pre-orden** del árbol que representa el término

```
SML
1  datatype tree = Leaf | Br of tree * string * tree;
2
3  val exprTree =
4  Br(
5  Br(
6  Br(Leaf, "x", Leaf),
7  "*",
8  Br(Leaf, "45", Leaf)
9  ),
10 "+",
11 Br(
12 Br(Leaf, "z", Leaf),
13 "/",
14 Br(
15 Br(Leaf, "y", Leaf),
16 "+",
17 Br(Leaf, "5050", Leaf)
18 )
19 )
20 );
21
22 fun unir ("", s) = s
23     | unir (s, "") = s
24     | unir (s1, s2) = s1 ^ " " ^ s2;
25
26 fun unir3 (a, b, c) = unir (unir (a, b), c);
27
28 fun preorden Leaf = ""
29     | preorden (Br(izq, dato, der)) =
30     unir3 (dato, preorden izq, preorden der);
31
32 preorden exprTree;
33
```

Output

```
> val Leaf = Leaf: tree;
> val Br = Br: tree * string * tree -> tree;
> datatype tree = {
  con Leaf = Leaf: tree;
  con Br = Br: tree * string * tree -> tree;
};
> val exprTree = Br (Br (Br (Leaf, "x", Leaf), "*", Br (Leaf, "45", Leaf)), "+", Br (Br (Leaf, "z", Leaf), "/", Br (Br (Leaf, -, -))): tree;
> val unir = fn: string * string -> string;
> val unir3 = fn: string * string * string -> string;
> val preorden = fn: tree -> string;
> val it = "+ * x 45 / z + y 5050": string;
```

- b) Construya el código en SML que tome una expresión de *tree* como parámetro y produzca la cadena que representa la serialización en orden del árbol que representa el término

```
datatype tree = Leaf | Br of tree * string * tree;

val exprTree =
  Br(
    Br(
      Br(Leaf, "x", Leaf),
      "*",
      Br(Leaf, "45", Leaf)
    ),
    "+",
    Br(
      Br(Leaf, "z", Leaf),
      "/",
      Br(
        Br(Leaf, "y", Leaf),
        "+",
        Br(Leaf, "5050", Leaf)
      )
    )
  );

fun unir ("", s) = s
  | unir (s, "") = s
  | unir (s1, s2) = s1 ^ " " ^ s2;

fun unir3 (a, b, c) = unir (unir (a, b), c);

fun enordenExacto Leaf = ""
  | enordenExacto (Br(Leaf, dato, Leaf)) = dato
  | enordenExacto (Br(izq, dato, der)) =
    "(" ^ enordenExacto izq ^ " " ^ dato ^ " " ^ enordenExacto der ^ ")";

enordenExacto exprTree;
```



Output

```
> val Leaf = Leaf: tree;
> val Br = Br: tree * string * tree -> tree;
> datatype tree = {
  con Leaf = Leaf: tree;
  con Br = Br: tree * string * tree -> tree;
};
> val exprTree = Br (Br (Br (Leaf, "x", Leaf), "*", Br (Leaf, "45", Leaf)), "+", Br (Br (Leaf, "z", Leaf), "/", Br (Br (Leaf, "-", "-))): tree;
> val unir = fn: string * string -> string;
> val unir3 = fn: string * string * string -> string;
> val enordenExacto = fn: tree -> string;
> val it = "((x * 45) + (z / (y + 5050)))": string;
```



- c) Construya el código en SML que tome una expresión de *tree* como parámetro y produzca la cadena que representa la serialización *post-orden* del árbol que representa el término

```
SML
1  datatype tree = Leaf | Br of tree * string * tree;
2
3  val exprTree =
4    Br(
5      Br(
6        Br(Leaf, "x", Leaf),
7        "*",
8        Br(Leaf, "45", Leaf)
9      ),
10     "+",
11     Br(
12       Br(Leaf, "z", Leaf),
13       "/",
14       Br(
15         Br(Leaf, "y", Leaf),
16         "+",
17         Br(Leaf, "5050", Leaf)
18       )
19     )
20   );
21
22  fun unir ("", s) = s
23    | unir (s, "") = s
24    | unir (s1, s2) = s1 ^ " " ^ s2;
25
26  fun unir3 (a, b, c) = unir (unir (a, b), c);
27
28  fun postorden Leaf = ""
29    | postorden (Br(izq, dato, der)) =
30      unir3 (postorden izq, postorden der, dato);
31
32  postorden exprTree;
33
```

```
Output
> val Leaf = Leaf: tree;
> val Br = Br: tree * string * tree -> tree;
> datatype tree = {
  con Leaf = Leaf: tree;
  con Br = Br: tree * string * tree -> tree;
};
> val exprTree = Br (Br (Br (Leaf, "x", Leaf), "+", Br (Leaf, "45", Leaf)), "+", Br (Br (Leaf, "z", Leaf), "/", Br (Br (Leaf, -, -)))): tree;
> val unir = fn: string * string -> string;
> val unir3 = fn: string * string * string -> string;
> val postorden = fn: tree -> string;
> val it = "x 45 * z y 5050 + / +": string;
```